Advanced Techniques for EEG Processing - Laboratory Exercises

Exercise 7 (this is the continuation of Exercise 1)

Exercise 1 applies some preprocessing steps to EEG data acquired on a subject while performing a trial-based experiment, consisting in a 3-stimulus auditory oddball task (see the text of Exercise 1 for the explanation of the task). The EEG data were recorded continously from 59 electrodes using CPz as reference electrode, at sampling rate of 500 Hz. The preprocessing steps in Exercise 1 consist of: i) detrending; ii) low-pass, high-pass and notch filtering; iii) extraction of epochs (in a 3-D matrix); iv) concatenation of extracted epochs (returning to a 2-D matrix); v) identification of bad channel(s) using the correlation method; vi) removal of bad channel(s) . Here, the preprocessing steps are completed by: computing ICA on good channels and cleaning good channels from artifacts; interpolating the bad channels using the cleaned good channel data; re-referencing to average reference also recovering the online reference electrode (CPz). Create a new .m file for the following steps.

1) Load the .mat file saved at the end of Exercise 1 `sub-035_PreprocessStep1.mat` containing the following quantities:

`X`: 2-D matrix of the epoch-concatenated data containing only good channels;

`ch_names`: cell array containing the labels of all the $n = 59$ electrodes (both good and bad electrodes);

`srate`: value of the sampling rate (500 Hz);

`stim_types`: cell array indicating the type of each applied stimulus (labels: 'standard', 'target', 'distractor');

`index_bad`: the numerical array containing the index/indices of the bad channel/s;

`index_good`: the numerical array containing the indices of the good channels **.**

Note that, only one channel (index 11, corresponding to F1) resulted 'bad' based on the adopted criterion. Therefore, `X` has size 58×97500 where 58 is the number of good channels and 97500 corresponds to the number of time samples of all concatenated epochs = total number of epochs $(195) \times$ number of samples per epoch (500, since each epoch contanins 1 second data).

2) Save the good channels data `X` in a .mat file (e.g., `dataExercise7_1.mat`).

3) For the data `X`, obtain the demixing matrix and the topographic maps of the ICs using the toolbox EEGLAB, through the following steps:

3.a) Launch EEGLAB from the command window `>>eeglab`

3.b) Import in EEGLAB the saved .mat file (obtained at point 2) and the file containing the good channel locations ('Standard-10-20-Cap58_good.locs'). To this aim, from the EEGLAB GUI select '*File>Import Data>Using EEGLAB Functions and Plugins>From ASCII/float file or Matlab array*', then from the 'Browse' button on the top select the saved .mat file and in correspondence to '*Data sampling rate'* indicates the value of the sampling rate (in Hz). In the same window, use the 'Browse' button on the row of '*Channel location file or info*' to import the file 'Standard-10-20-Cap58_good.locs'. Leave unfilled the other edit boxes. Press OK and assign a name to the dataset. It is possible to display the time pattern of the imported signals '*Plot>Channel data (Scroll)*'.

3.c) Estimate the demixing matrix and the independent components of the imported dataset by selecting '*Tools>* D*ecompose Data by ICA*' and setting

ICA algorithm to use → *runica*

Commandline options → 'extended', 1, 'bias', 'off', 'rndreset', 'no'

Press Ok and wait for the algorithm to terminate. It is possible to display the time pattern of the estimated independent components selecting '*Plot>Component activations (Scroll)*'.

3.d) Display the topographic maps of the estimated independent components (limit to only the first 30 components) selecting '*Plot>Component maps>In 2-D*' and type *1:30* in correspondence to '*Component numbers'*). Save the (.fig) containing the topographic maps (e.g. `mapICs_Exercise7.fig`).

3.e) Export the demixing matrix selecting '*File>Export>Weight matrix to text file'*, and give a name to the file, specifying the extension .txt (e.g. `matrixW_Exercise7.txt`).

3.f) Save the current dataset created in EEGLAB selecting '*File>Save current dataset(s)*', assigning a name (two files are automatically saved with extension .set and .fdt).

3.g) Close the EEGLAB windows and clean the workspace (`clear`).

4) Return in your m. script and use function `load` to import the demixing matrix from the saved file. Use the demixing matrix to compute the estimated independent components and plot them as a function of time.

5) Compute the power spectral density of the estimated independent components (using `pwelch`).

6) The independent components that are likely to mainly extract artifact activity are: 1,4,6,11, from 14 to 25, 28 and 29. Using function `plot_IC.m`, examine the spatial, spectral and time pattern of some of these independent components, e.g. 1,4,6,11,15,20,23 (and compare with other non-artifact ICs such as 5,7,9). Note that in function `plot_IC.m` you have to use the locs file 'Standard-10-20-Cap58_good.locs' since the ICs have been estimated for the dataset of 58 good channels.

7) Remove the artifact ICs (1,4,6,11, from 14 to 25, 28 and 29) and reconstruct the good channels data cleaned from the artifact components. Plot the time pattern of the good channels before and after artifact removal.

8) Add back the bad channel to the dataset of cleaned good channels. To this aim, add a row of all zeros at the original bad channel position (row = 11, as stored in `index_bad`) obtaining a set of 59 signals. Note that the values at the bad channel are irrelevant since in the following step the bad channel will be interpolated based only on the values at good channels. Save the so obtained dataset (59×97500) in a .mat file (e.g. `dataExercise7_2.mat`); this will be imported in EEGLAB to interpolate the bad channel (see point 9).

9) Open EEGLAB and import the .mat created at point 8, together with the file of channel locations 'Standard-10-20-Cap59.locs' (now you have 59 channels, where the 11 one is bad and has to be replaced via interpolation), and insert the value of the sampling rate. After importing, select *'Tools>Interpolate Electrodes'*. In the window, under 'What channel(s) to you want to interpolate', use 'Select from data channels' and choose the channel (F1) to be interpolated. As 'Interpolation method', leave 'Spherical'. Press Ok. Then, in the Command Window, run the instruction `Xinterpolated=double(EEG.data);` (EEG.data is the dataset in EEGLAB containing the 59 channels with the interpolated F1 channel; `double` convert from single to double precision). Finally, from the Command Window save the variable `Xinterpolated` in a .mat file (e.g. `dataExercise7_3.mat`).

10) In your .m script, load the .mat file generated at point 9 (`dataExercise7_3.mat`), containing all good channels + interpolated bad channel(s). Perform re-referencing to average reference, adding back the signal at the online reference electrode (CPz) in the last row. Append the label 'CPz' to the cell array `ch_names`.

11) The final data obtained at point 10 are organized as 2-D matrix having size 60×97500, and containing all cleaned good chans + interpolated bad chan(s) + recovered online reference CPz, all referred to average reference. Now convert the data to a 3-D matrix having the channels ($n = 60$ now) along the first dimension, the samples within each epoch ($m_ep = 500$) along the second dimension, and the epochs along the third dimension. The overall number of epochs (q) is equal to the length of `stim_types`. To convert the data from 2D to 3D, use `reshape`.

Finally, save in the .mat file `sub-035_PreprocessedStep2.mat` the following quantities

`X`: 3-D matrix of the data after all the previous preprocessing steps (size $n \times m_ep \times q$)

`ch_names`: cell array containing the labels of all $n = 60$ electrodes;

`srate`: value of the sampling rate;

`stim_types`: cell array indicating the type of each applied stimulus (labels: 'standard', 'target', 'distractor').

OPTIONAL: Apply all preprocessing steps (Exercise 1 + Exercise 7) to the EEG data of a different subject involved in the same experiment "`sub-003_ses-01_task-Rest_eeg.mat`", using the same script files already used in Ex1 and Ex2 for sub-035 (note that by applying the correlation criterion, you will find no bad

channel for sub-003, and points 8 and 9 in Ex7 will be skipped for this subject). Note that in this case, the independent components that are likely to mainly extract artifact activity are: 1, 2, 3, 11,from 14 to 17, 19, 20, 22, 25, 27, 28, 30.

Function

```
[P,F]=pwelch(X,WINDOW,NOVERLAP,NFFT,Fs)
```

returns in `P` the power spectral density of `X`, estimated by dividing `X` into sections containing a number of samples specified in `WINDOW`. `NOVERLAP` specifies the samples of overlap between adjacent sections (leaving it at the default value [ ], a 50% overlap is used). `NFFT` is the number of points used to calculate the spectrum; if `NFFT` is greater than `WINDOW` the data in each section is zero-padded. `Fs` is the sampling frequency (= srate). `F` returns the vector of frequencies at which the power density is estimated (the resolution in Hz is equal to Fs/NFFT). Specifically, `F` is a $n_f \times 1$ vector of frequencies from 0 to $Fs/2$ with step $Fs/NFFT$ (thus $n_f = \frac{\text{NFFT}}{2} + 1$)

If `X` is a vector, it is converted to a column vector and treated as a single channel. If `X` is a matrix ($m \times n$), the power spectral density is calculated independently for each column and saved in the corresponding column of `P` (size of `P` = $n_f \times n$).

<u>Suggestion for computing PSD</u>: use `WINDOW` of 5 seconds (`WINDOW=5*Fs`), NOVERLAP at the default value (`[]`), and use zero-padding to increase frequency resolution (`NFFT=2*WINDOW`). The frequency resolution is thus $F_s/\text{NFFT} = 0.1$ Hz.

Function (not built-in function)

```
plot_IC(index_ICs,A,Y,P_ICs,F,chan_locs,Fs,t1,t2)
```

for each selected IC (indicated in `index_ICs`, for example `index_ICs=[1 2 6])` generates a single figure containing the topographic map of the IC, the time pattern and the power spectral density. `A` is the estimated mixing matrix (= inv(W), where W is the demixing matrix obtained from EEGLAB), `Y` ($n \times m$) is the overall set of estimated independent components (not only the ones to be plotted), `P_ICs` ($n_f \times n$) is the set of power spectral density of all ICs (returned by applying `pwelch` to Y'), `F` ($n_f \times 1$) is the vector of frequencies returned by `pwelch`, `chan_locs` is the name (between quote marks) of the .locs file, `Fs` is the sampling frequency, `t1` and `t2` denote the time range (in s) for plotting the IC time pattern.

e.g.

```
t1=0; % s
t2=30; % s
index_ICs=[1 2 6];
plot_IC(index_ICs,A,Y,P_ICs,F,'Standard-10-20-Caps19.locs',srate,t1,t2)
```